

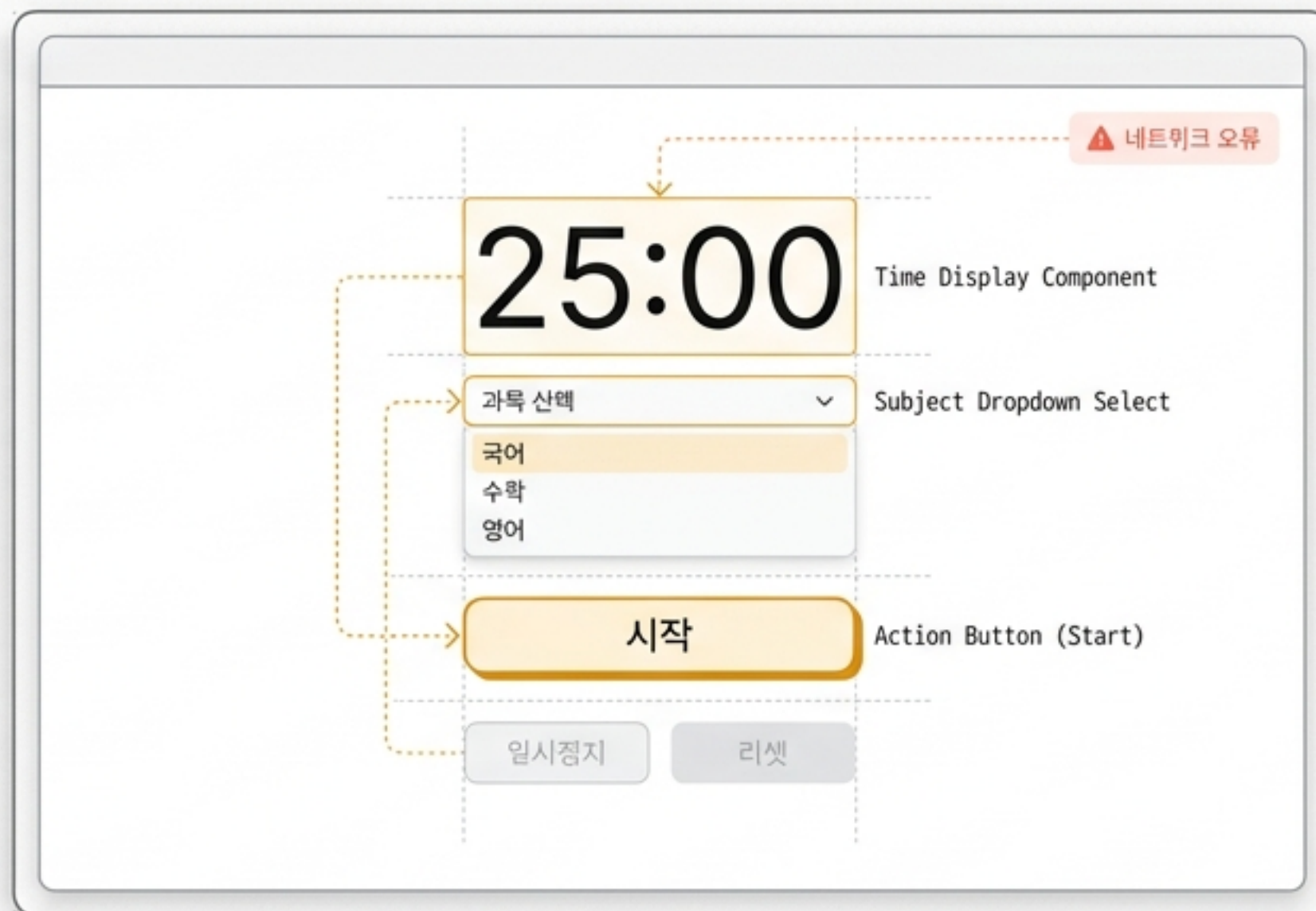
바이브 코딩으로 완성하는 나만의 공부 타이머

중학생 대상 2시간 프론트엔드 완성 프로젝트 (Day 2)

Human Intent Input

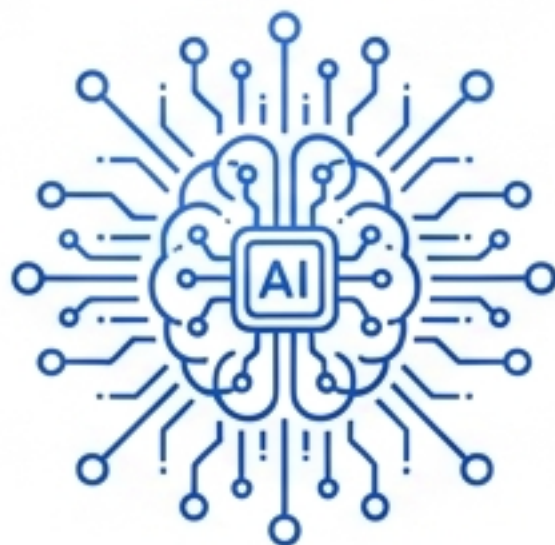
과목별로 시간을 썰 수 있는 심플한
공부 타이머를 만들어줘.

→ AI Prompt



코딩의 새로운 패러다임: AI가 짜고, 인간이 검증한다

Teacher (AI Pilot)



- ◆ 프롬프트 설계 & 코드 생성
- ◆ 선생님이 자연어로 요구사항을 입력하면 AI가 즉시 코드를 작성.

Student (QA & Developer)

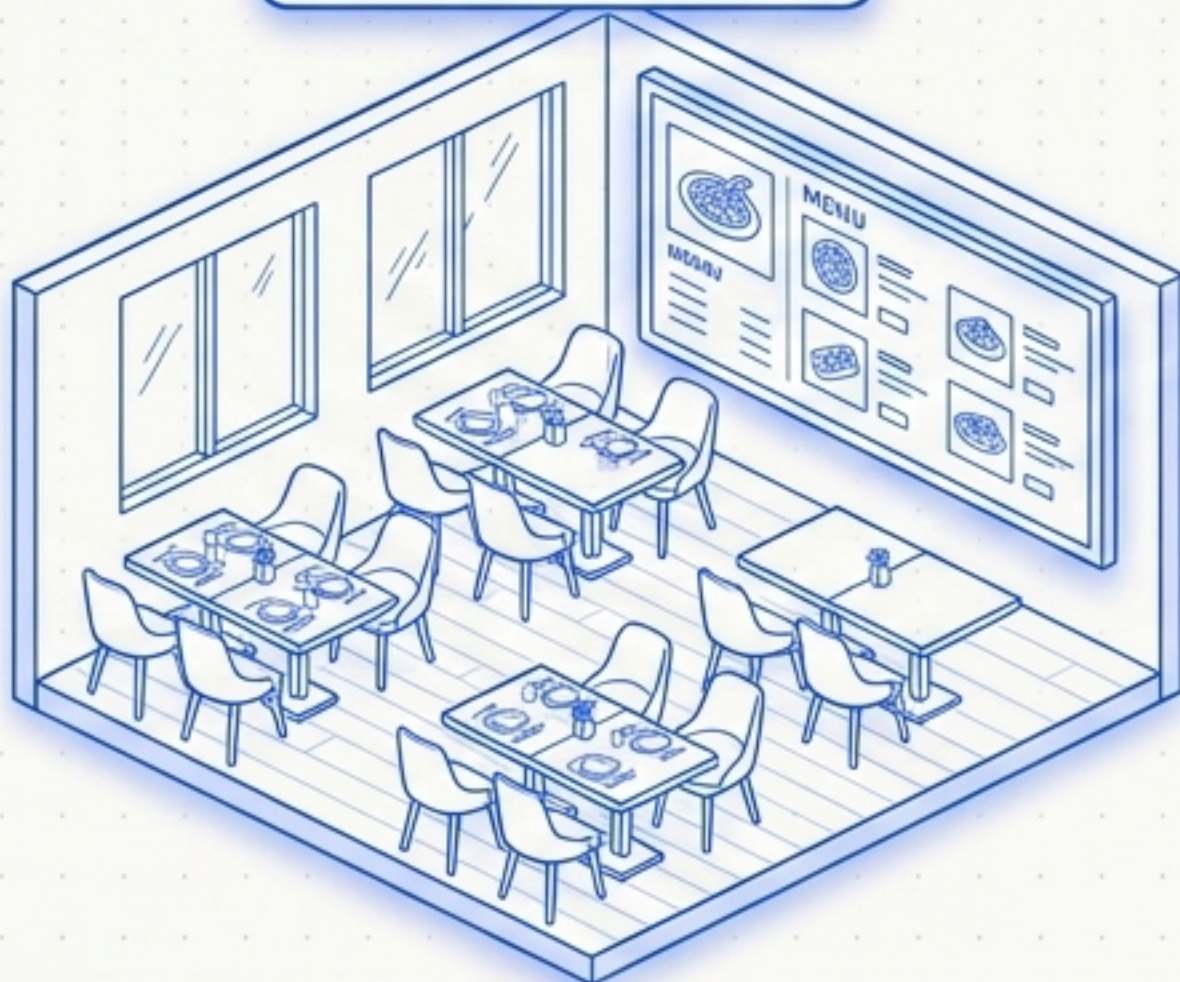


- ◆ 로컬 실행 & UI/UX 검증
- ◆ 학생은 코드를 VSCode에 붙여넣고 브라우저에서 직접 테스트하며 수정 사항을 역제안.

목표는 문법 암기가 아닌, '개발 → 테스트 → 디버깅'이라는 실제 소프트웨어 개발 사이클의 압축적 체험입니다.

오늘의 개발 범위: 주방 없이 오픈하는 레스토랑

Frontend (오늘의 목표)



인테리어와 메뉴판 (HTML/CSS/JS)

사용자가 직접 보고 클릭하는 화면. '더미 데이터(Dummy Data)'를 사용해 가짜 메뉴로 화면을 먼저 테스트합니다.

Backend (다음 시간)



주방과 냉장고 (Python/Flask + Database)

진짜 데이터를 저장하고 처리하는 서버.

백엔드(주방)가 없어도 프론트엔드(메뉴판)를 먼저 설계하고 시각화할 수 있습니다.

UX 파운데이션 1: '누가' 쓸 것인가? (페르소나 정의)

페르소나 부재 시

'무슨 기능을 넣지?'
→ **방향성 상실** (기능의 과잉)

페르소나 존재 시

'시험 기간의 중학생에게
필요한 것은?'
→ **명확한 개발 로드맵**

페르소나: 민수 (중학생)

Worries

과목별로 시간을
재고 싶어

오늘 얼마나 공부했는지
한눈에 보고 싶어

복잡한 건 싫어,
바로 누르고 싶어

Features

필수 기능:
과목 선택 드롭다운

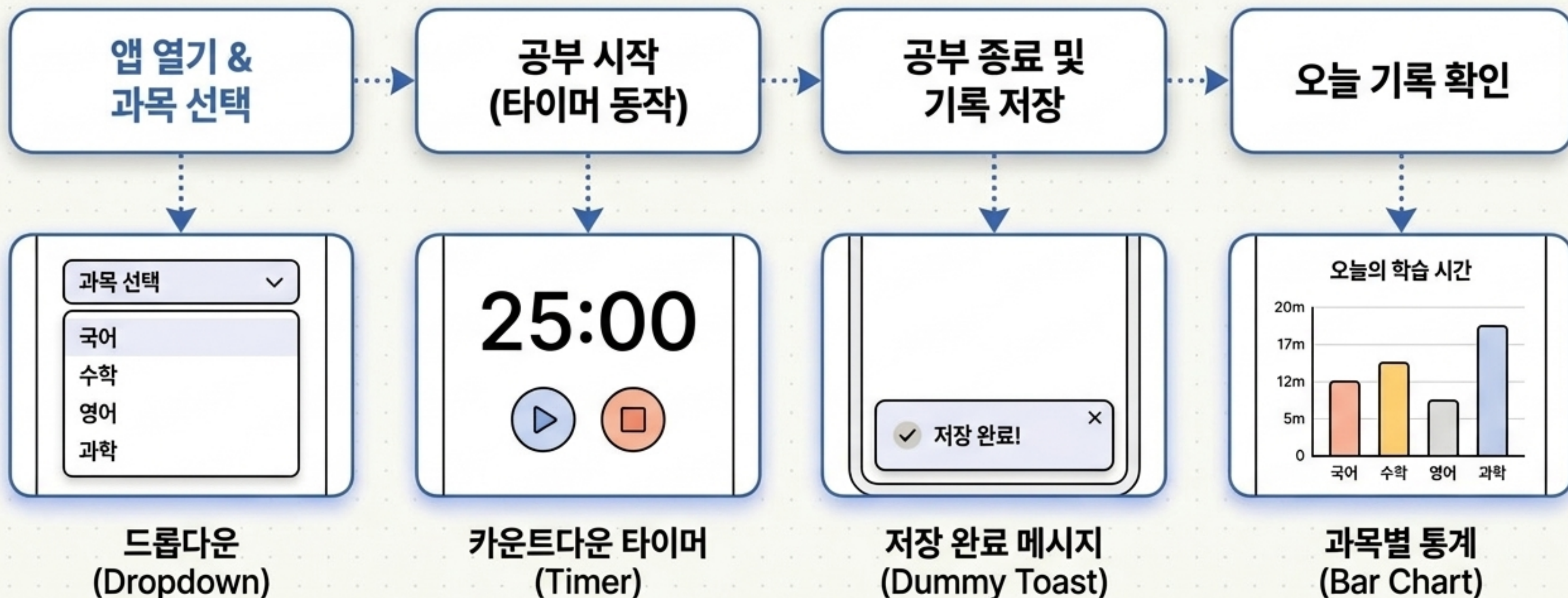
필수 기능:
공부 기록 막대 그래프

필수 기능:
심플한 대형 타이머 UI

UX 파운데이션 2: 기존의 거인들에게서 배우다 (벤치마킹)

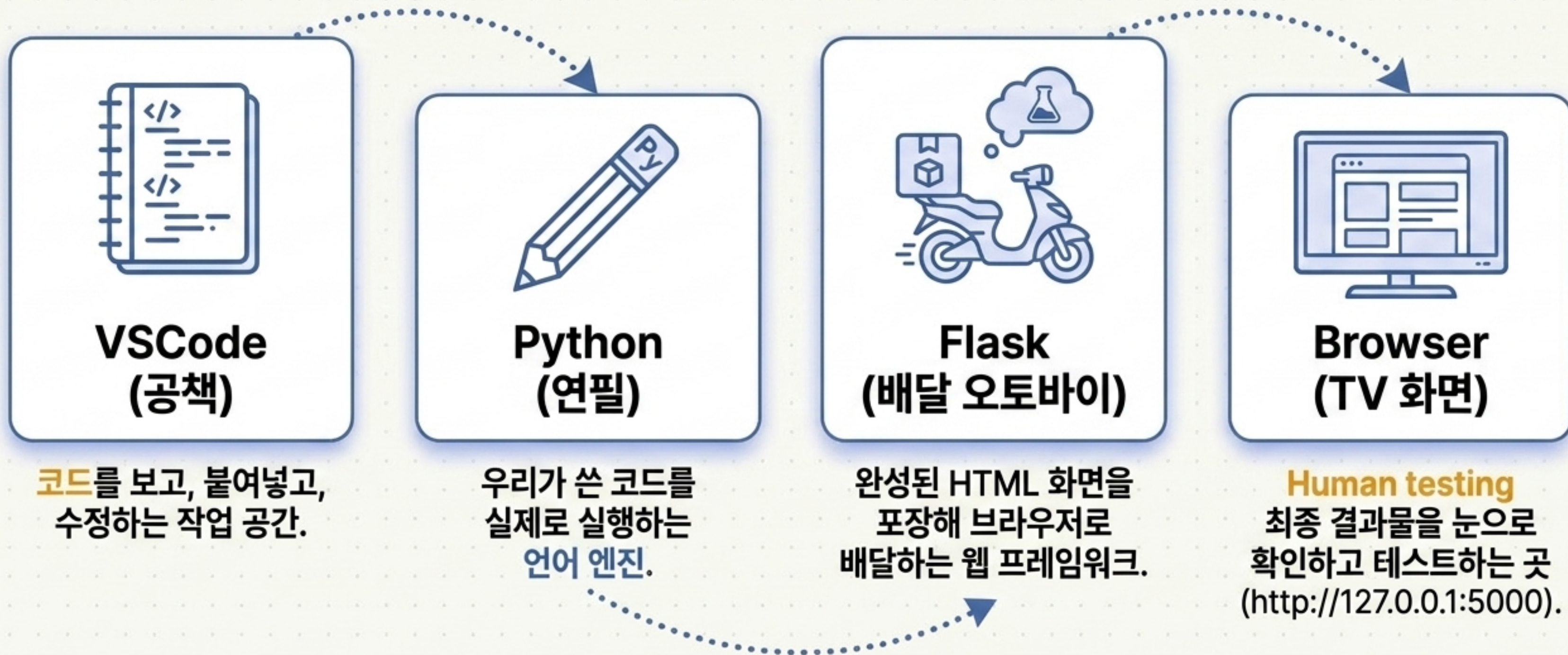
앱 (App)	장점 (Pros)	아쉬운 점 (Cons)	우리의 전략 (Strategy)
뽀모도로 타이머	25분/5분 자동 반복 (효율적 리듬)	과목 구분이 없음	과목별 타이머 기능 추가
Forest (포레스트)	나무가 자라는 게임 요소 (흥미 유발)	즉각적인 공부 기록 확인이 불편함	대시보드에 즉시 기록 막대 그래프 표시
열품타 (열정품은타이머)	정확한 순공시간 측정	화면과 기능이 너무 복잡함	핵심 기능만 남긴 미니멀리즘 디자인

UX 파운데이션 3: 시간의 흐름을 화면으로 변환하다

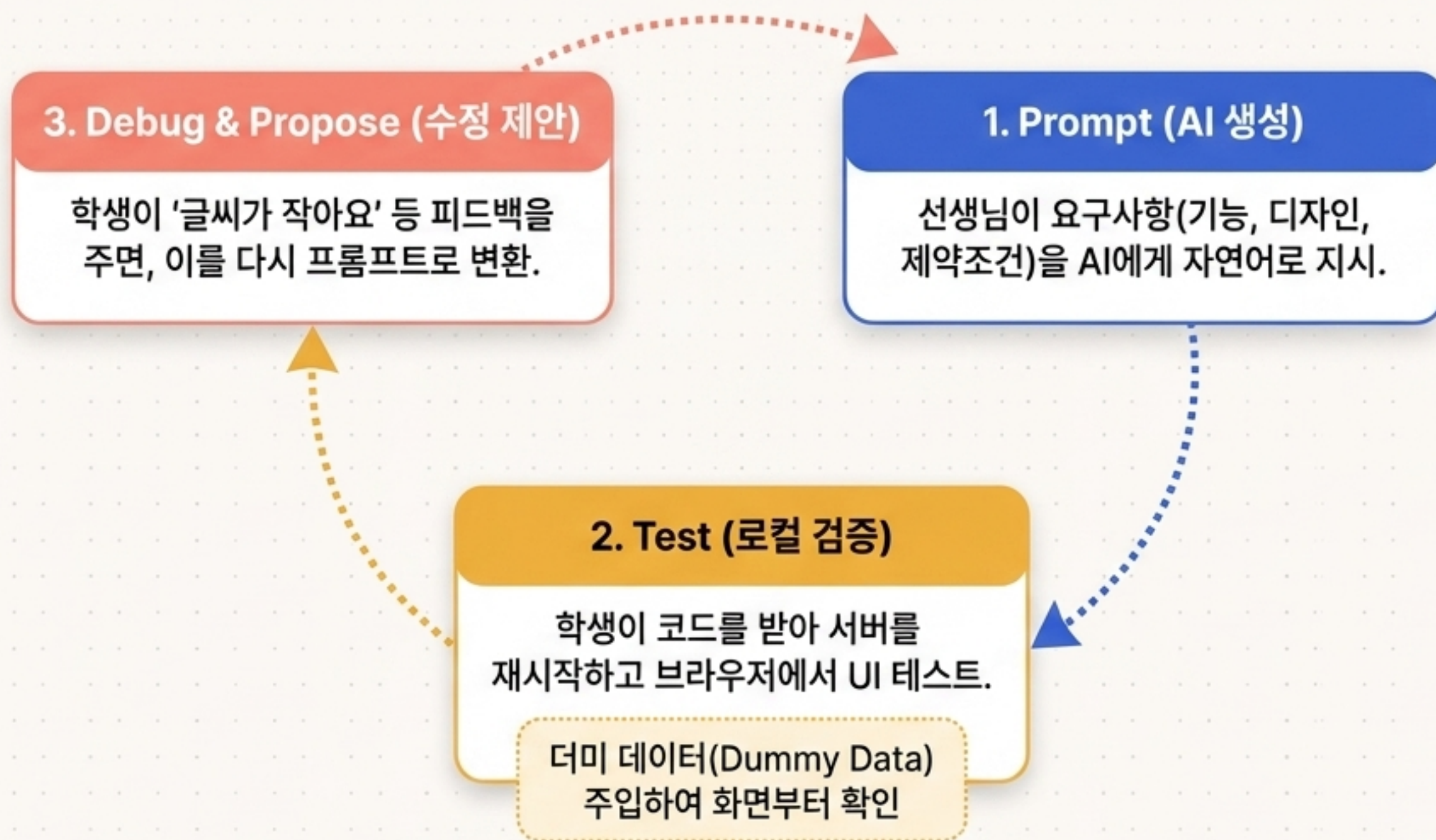


유저 시나리오의 한 단계는 곧 우리가 오늘 만들어야 할 '화면 블록' 하나를 의미합니다.

로컬 개발 생태계: 내 컴퓨터를 서버로 만들기



바이브 코딩 엔진: 무한 피드백 루프



오늘 2시간 동안 이 엔진을 2회전(과목 선택 추가 → 기록/통계 추가)시켜 프론트엔드 앱을 완성합니다.

사이클 1: 자연어를 UI 컴포넌트로 번역하다

Before & After

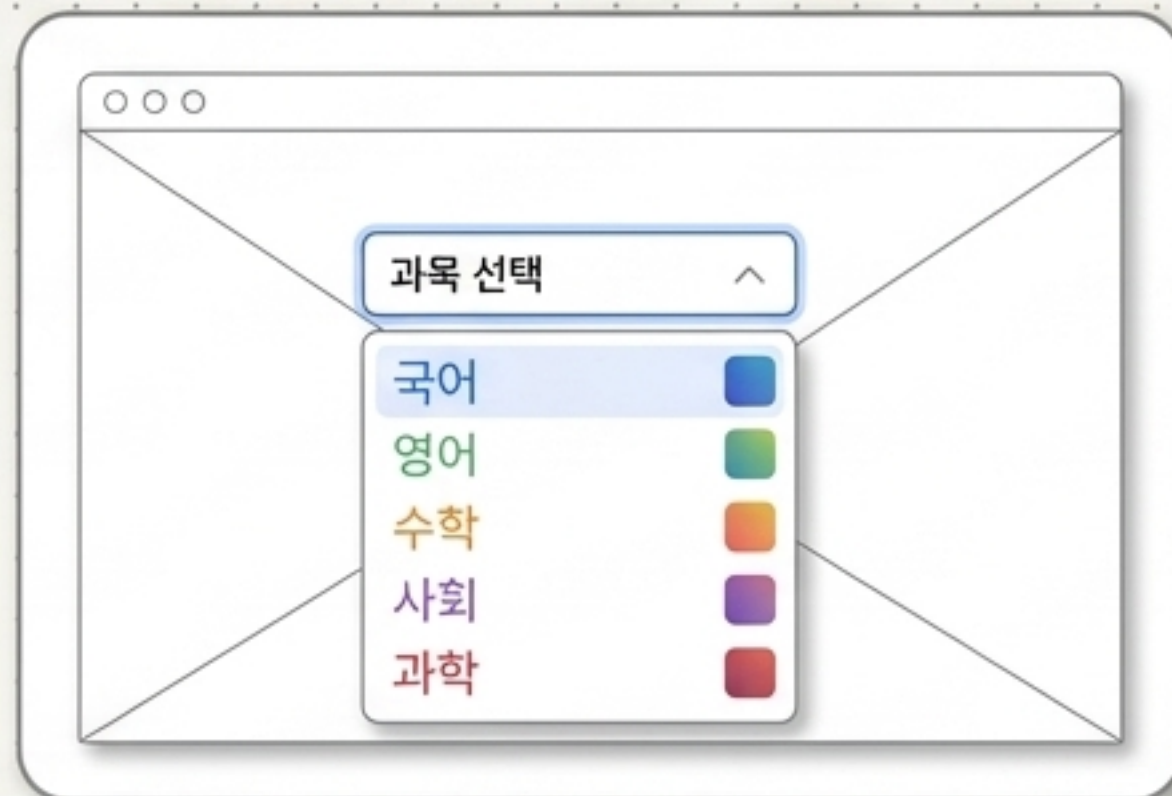


The Prompt

- 과목을 선택할 수 있는 드롭다운(select)을 만들어줘.
- 과목 목록: 국어, 영어, 수학, 사회, 과학 (5개).
- 과목마다 텍스트 색깔을 다르게 해줘. 프론트엔드만 구현해.

Claude Code
HTML/CSS Generation

The Result



학생 피드백 연습



학생의 직관



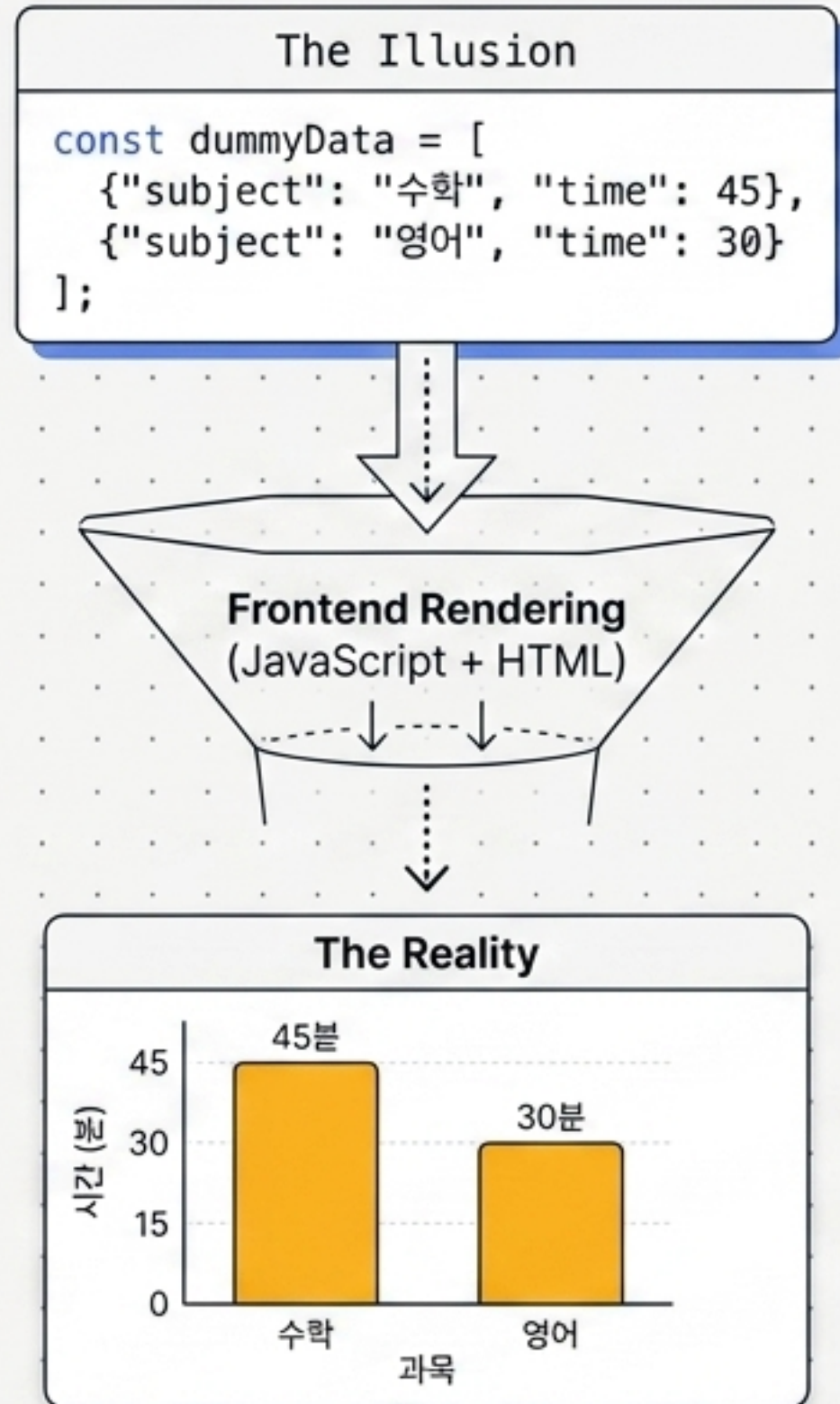
글씨가 너무 작아요!



프롬프트 엔지니어링

`</>` 과목 이름의 font-size를 30px로 키워줘.

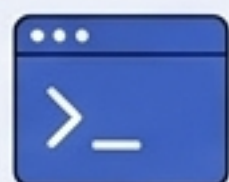
사이클 2: 더미 데이터(Dummy Data)의 마법



핵심 개념 (Key Concept)

주방(DB)이 아직 없어도, JavaScript 변수에 '가짜 데이터'를 넣어두면 완벽하게 작동하는 통계 대시보드 화면을 미리 완성하고 테스트할 수 있습니다.

디버깅 트리아지(Triage): 에러는 실패가 아닌 단서다



서버 에러 (Server Error)

증상:

터미널에 뜨는 빨간 글씨
(예: `ModuleNotFoundError`)

→ 해결책:

코드 문제. 에러 메시지를 복사해
AI(선생님)에게 그대로 전달.



화면 에러 (Screen Error)

증상:

브라우저 UI가 깨지거나 버튼이
엉뚱한 곳에 배치됨

→ 해결책:

브라우저 F12 (Console) 확인, 혹은
에디터에서 **Ctrl + Z**로 되돌리기.



기능 에러 (Logic Error)

증상:

버튼을 눌러도 아무 일도
일어나지 않음

→ 해결책:

'A를 눌렀는데 B가 안 돼요'라고
인과관계로 명확히 설명하기.

! Banner Alert

학생 개발자의 가장 강력한 무기: **Ctrl + Z** (실행 취소)와 강력 새로고침 **Ctrl + Shift + R**

프롬프트 엔지니어링 공식: AI를 정확하게 조종하는 법

초기 뼈대 구축

[목표: 공부 타이머] + [기술 스택: Flask, HTML] + [파일 구조: app.py, index.html] = 기본 뼈대 완성

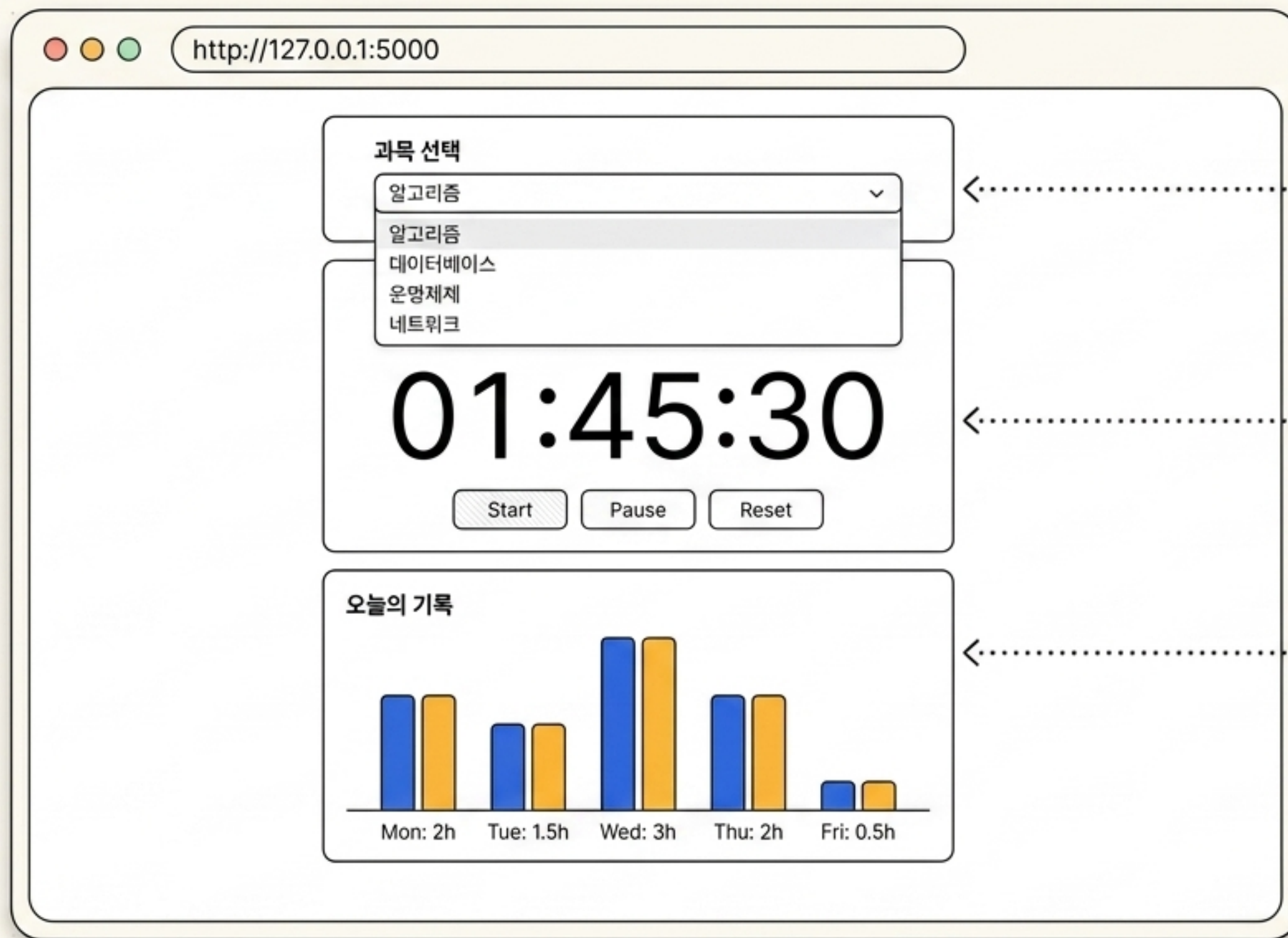
정밀한 기능 확장

[기존 기능: 타이머] + [새 기능: 초기화 버튼] + [위치: 정지 버튼 옆] = 충돌 없는 기능 추가

오류 및 범위 초과 방지 (가장 중요한 제약 조건)

"프론트엔드만 구현해." + "백엔드 연동 없이 더미 데이터 사용해." = 정확한 스코프 제어

2시간의 결과물: 완성된 프론트엔드 대시보드



페르소나 맞춤형 과목 선택
(더미 데이터 연동)

직관적이고 심플한 카운트다운 타이머
(JavaScript 구동)

오늘의 기록 막대 그래프
(시각적 피드백 완료)

페르소나 정의부터 UI 테스트까지.
완벽하게 작동하는 '메뉴판'이 완성되었습니다.

Next Steps: 주방(Backend)에 불을 켜 시간

Today - Day 2



Day 2 완료: 완벽한 인터리어와 작동하는 메뉴판 구축

Next Class - Day 3



Day 3 예고: 실제 데이터베이스 연결 및 진정한 백엔드 API 연동!

다음 시간에는 껌데기뿐인 식당에 진짜 요리사를 고용하고 주방을 가동합니다.